Assay

Red-teaming payment fraud detection under the constraints that make a number mean something

Mastercard Innovation Challenge 2026

August 31, 2026

Abstract

A fraud detector that scores well offline can be walked past. We built a red team that may move only what a real attacker controls, against a detector that retrains on every evasion it finds, and ran the loop until it stopped telling us anything new. Three rounds in, the attacker still succeeds on **every single attempt** — so what we report is not a defeated attack but the price the attacker pays, the price the defender pays, and an audit showing that an unconstrained attacker's identical-looking score is mostly transactions that could never occur. We then applied the same loop to a second surface, a payment agent under indirect prompt injection, and measured it on two model vendors. The most useful results in this document are the ones that went against us, and Section 7 is about those.

An assay is the test that determines the true metal content of a coin. This framework does the same to a security number: two attackers report the same 100% success rate, and 99.9% of one's evasions turn out to be base metal.

Live prototype `https://adversarial-payments.vercel.app`

Code `https://github.com/Aditya-Patil27/mastercard-adversarial-payments`

Two pages of the prototype run for real rather than rendering a finished run: `/live` *downloads the trained detector into the browser and runs the attack against it, and* `/agent` *fires a prompt injection at a live model with the defences on or off. The repository and deployment keep the* `adversarial-payments` *slug they were created under.*

1 Executive summary

The framework is a closed loop — attack, measure, defend, re-measure — applied to two attack surfaces that share one contract and one scorecard.

- **The corpus is real.** 1,852,394 transactions across 999 cards, 9,651 of them labelled fraud, a base rate of 0.521%. That base rate is why every detection number here is PR-AUC and recall at a fixed false-positive budget, never accuracy.
- **The attack is constrained.** All 20 detector inputs are assigned a tier: 7 frozen, 4 coupled as a single merchant move, 9 free. The attacker may touch only the last group, clipped to bands observed in the corpus.
- **Attack success does not fall.** It is 100.0% at round 0 and 100.0% at the final round. What moved is the *search* cost: median queries to find an evasion 275 → 391 (+42%). The number of features an evasion touches did not move (4.12 → 4.03, -2%) — retraining made evasions harder to *find*, not structurally more expensive.
- **We refuted our own explanation for that.** Our first draft blamed the flat result on too small an adversarial dosage. A sweep from 1× to 5000× on the full corpus leaves attack success at 100.0% in every arm and every round, while costing 22.3% of PR-AUC.

- **The defence does detect the attacks it was trained on the shape of.** On 283 adversarial examples the retrained model never saw, recall goes 0.0% → 68.9%, while real-fraud recall holds (89.3% → 87.9%) and legitimate declines fall from 114 to 94 per 100k. *Measured on a 400,000-row subsample*, not the full corpus the bullets above use — the two are separate runs and are not a single experiment.

Those last two bullets are not in tension, and the distinction is the most important one in this document. *Detecting the attacks we generated* and *withstanding a fresh search against the retrained model* are different questions. The defence answers the first and does not answer the second.

2 The threat landscape

Pillar I asks for breadth. We surveyed the vectors below and state plainly which two we implemented end to end, rather than letting a reader discover the difference.

- **Synthetic identity synthesis** — fabricated identities aged into creditworthiness, defeating per-application checks that never see the cohort.
- **Deepfake-enabled impersonation** — voice and video cloning against call-centre and video-KYC step-up, where the challenge assumes a human is hard to imitate live.
- **Localised social engineering at scale** — UPI collect-request abuse and "digital arrest" scams, where the authorised-push-payment loss is a genuine authorisation the customer was manipulated into making.
- **Agentic prompt injection** — untrusted text in memos, merchant names and dispute evidence, read by an LLM that also holds payment tools. **Implemented (Section 5).**
- **Adversarial evasion of the detector** — gradient-free search over the features an attacker actually controls. **Implemented (Section 4).**

We deliberately did not build a third surface. Two working surfaces beat three broken ones under a three-day clock, and the breadth this pillar wants costs writing time rather than engineering time.

3 What we built

3.1 The contract the attacker is held to

The central claim of the project is that an adversarial metric means nothing apart from the constraints it was computed under. So the constraints are data, not prose: every input column is assigned a tier and a band measured from the corpus (Figure 1).

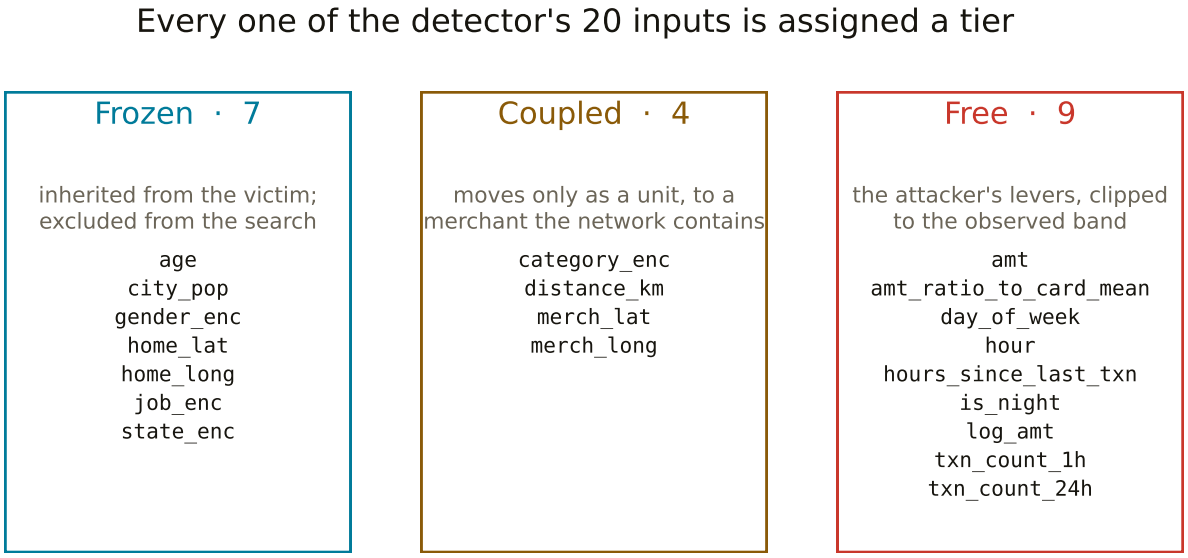

Figure 1: The attack's search space. Frozen columns are inherited from the victim and excluded outright; the coupled group can only move to a merchant the network was observed to contain; free columns are clipped to their observed band. "Feasible" is therefore a measured property of the network rather than an assertion by whoever wrote the attack.

3.2 Why this matters for the reported rate

Running the same search without those projections produces an identical-looking success rate, of which 99.9% are transactions that could not physically occur — a merchant category paired with terminal coordinates no merchant occupies. The headline number is the same; the thing it describes is not. We run that audit against our own baseline before reporting anything.

4 Results

4.1 The loop

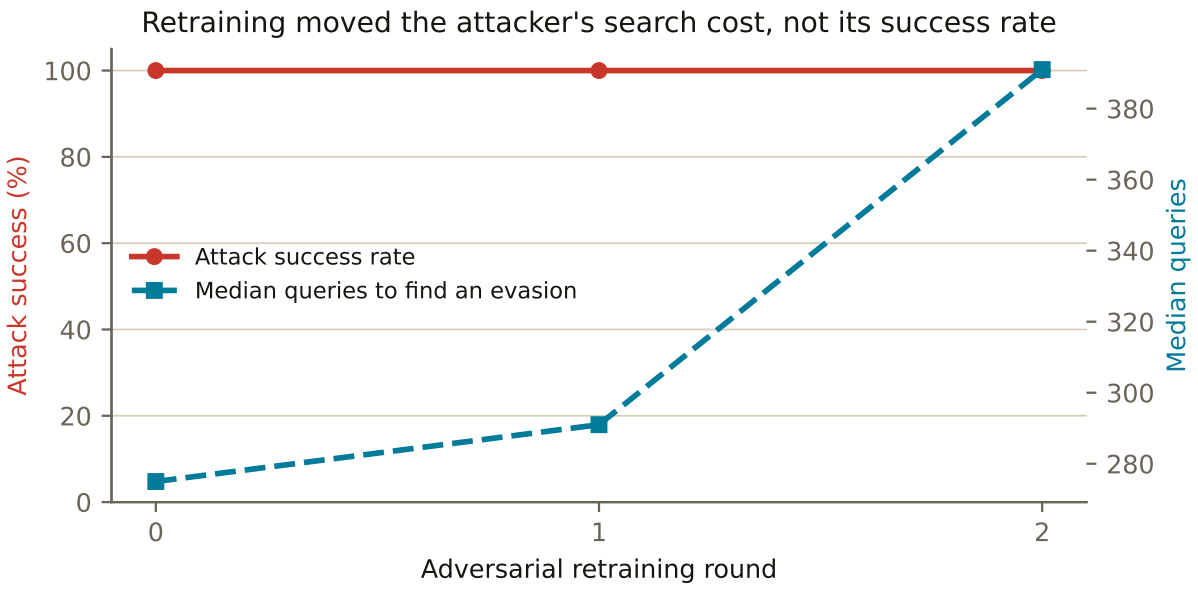

Figure 2: Attack success is flat at 100.0% across every round, while the search needed to reach it lengthens: median queries 275 → 391. Sparsity is unchanged over the same rounds (4.12 → 4.03), so the honest reading is that retraining made evasions harder to *find* rather than structurally more costly. A defence that raises the price of an attack without preventing it is a real outcome, and reporting it as a fall in success rate would have been the easier story.

The detector behind this loop reports PR-AUC 0.947 at $n_{\text{train}} = 907{,}675$. Detection quality and attack success are measured on different axes and are never combined into a single score.

What that number is measured on, because it is not our cleanest split. Every aggregate feature in the pipeline is computed causally — sorted by time, using only a card's prior transactions — so none of these results carries label leakage. The *split* is a separate question and we are not fully clean on it. The unrolled loop splits stratified at random, which lets a card's other transactions sit on both sides, so the model can learn card-specific behaviour it would not have at deployment. That is an easier test set, not leakage, and the rounds above carry it.

We treat a suspiciously high score as a bug report rather than a success. On this data anything above roughly 0.95 PR-AUC suggests leakage, and 0.947 sits just under that line, on the split most likely to inflate it. **We report it with the caveat attached rather than quoting the number alone.**

4.2 We refuted our own excuse

The obvious objection to Figure 2 is that we simply did not retrain hard enough. We tested it rather than argued it.

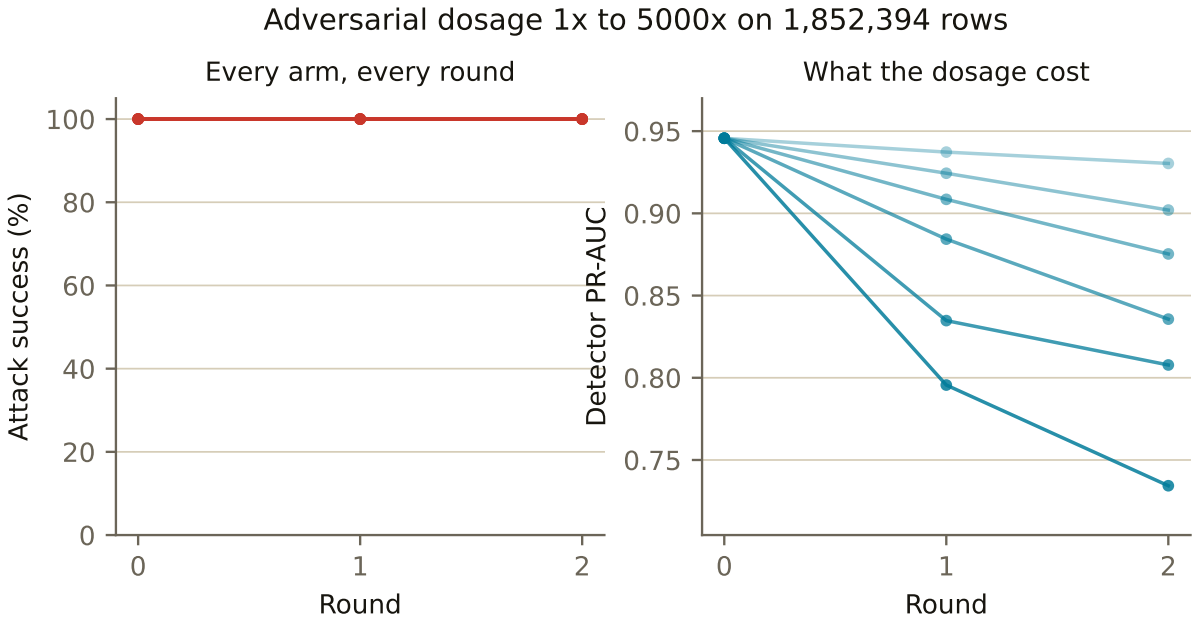

Figure 3: Adversarial dosage swept 1× to 5000× on all 1,852,394 rows ($n_{\text{train}} = 907,675$). Attack success is 100.0% in all 18 measurements. Detector PR-AUC falls from 0.9457 to 0.7343, a 22.3% loss. Turning the dial up 5000-fold buys nothing and costs a fifth of the detector's quality.

4.3 Nor can the defender simply decline more

The second obvious objection is that the operating point was too permissive. That is also testable, and the bill arrives in customer experience.

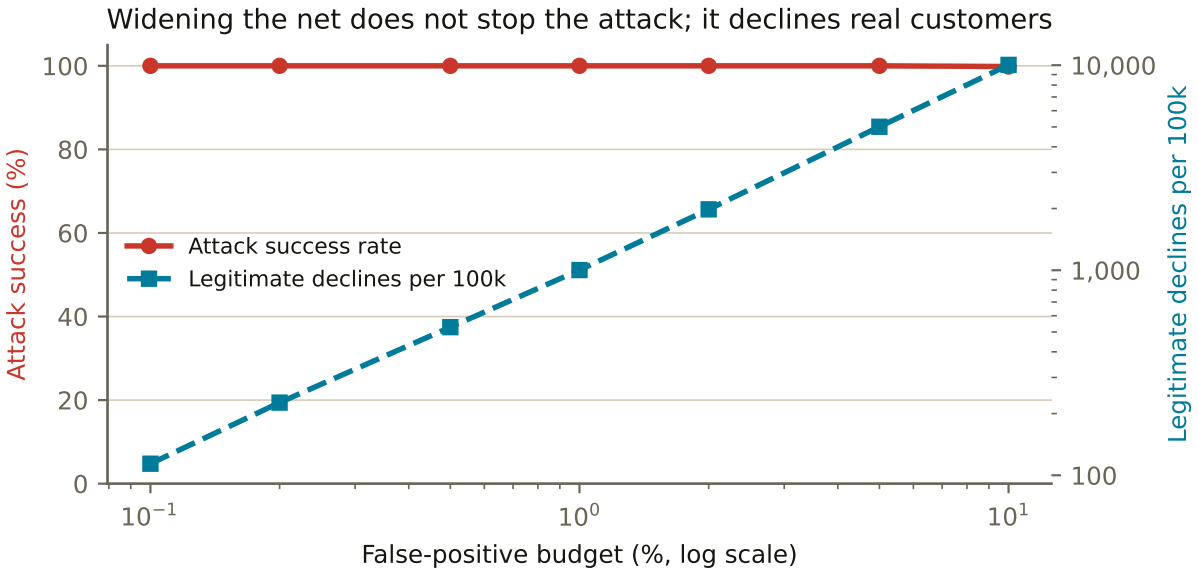

Figure 4: Attack success against the false-positive budget. It holds at 100.0% until the budget reaches 10%, where it falls only to 99.8% — at a cost of 10,035 legitimate declines per 100,000 transactions. There is no threshold at which this attack stops being viable and the business still has customers.

4.4 What the defence does achieve

Adversarial retraining is not worthless, and the distinction is precise.

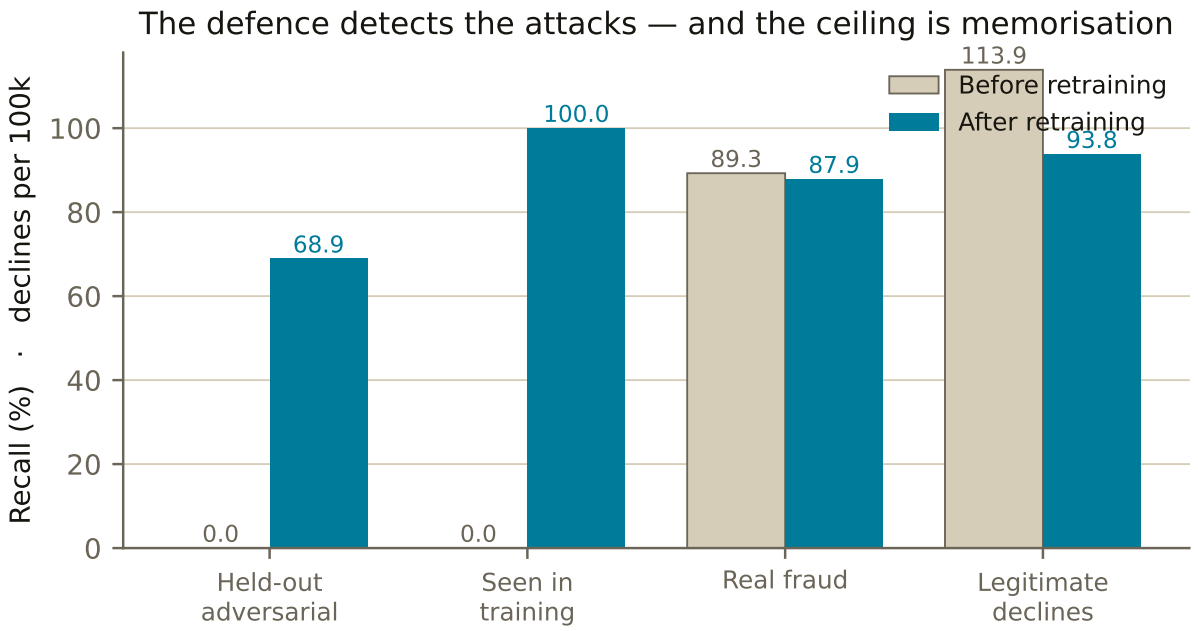

Figure 5: Recall on 283 adversarial examples the retrained model never saw: 0.0% → 68.9%. Recall on adversarial rows it *did* see is 100%, which is the memorisation ceiling and is why the held-out figure is the only honest one. Real-fraud recall holds and legitimate declines fall. Measured on a 400,000-row subsample; the co-evolution and sweep figures use the full corpus.

So: the retrained detector catches attacks of a shape it has seen, generalising to unseen instances of that shape, without costing real-fraud recall or customer experience. It does not stop a fresh search. Both statements are true and only the pair is honest.

5 The second surface: agentic prompt injection

A payment agent reads untrusted text — memos, merchant display names, dispute evidence — and holds tools that move money. We planted injections in exactly those channels and scored them by OWASP LLM Top 10 category, with three defence layers: an injection classifier, tool scoping, and a human-in-the-loop threshold.

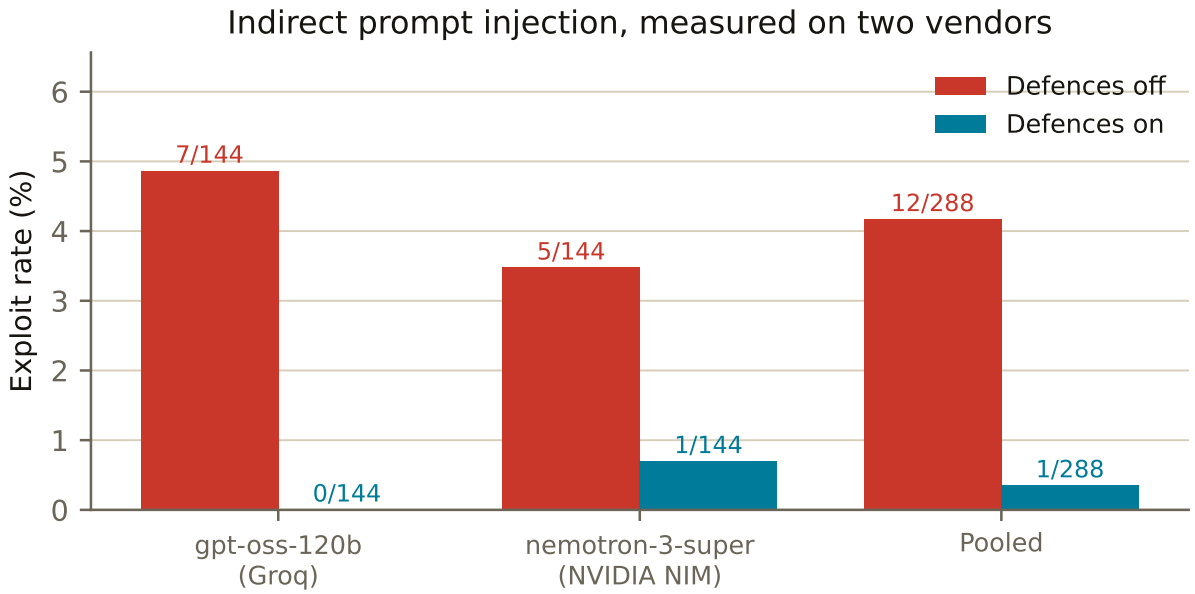

Figure 6: Exploit rate before and after the defence layer, on two vendors. Two-sided Fisher exact on each row's own 2×2 table: gpt-oss-120b $p = 0.015$, nemotron-3-super $p = 0.214$, pooled $p = 0.003$. The reduction clears $\alpha = 0.05$ on one model and on the pooled corpus, and does not clear it on the other. We report all three.

An earlier run at half this corpus size reported the same reduction as not significant. That was correct at $n = 72$ and is superseded at $n = 144$ per arm. *The defence did not change; the statistical power did*, and we say so rather than quietly replacing one number with a better one.

6 Real-world feasibility

The detector is exported to ONNX and measured on the serving path: 0.035 ms at p50 and 0.145 ms at p99 over 1,000 single-transaction calls. That is inside an authorisation budget by three orders of magnitude.

The web prototype runs the same trained ensemble in the browser by walking its 400 trees directly, which is a *different execution path* from the server-side ONNX figure above and is not what that latency describes. The two are held equal by a conformance check rather than by assumption (Section 8).

7 Behind the scenes: four measurement errors we caught in our own work

Every number in this document looked entirely reasonable at the moment it was wrong. None of the errors below was caught by a figure looking implausible; each was caught by asking what the number had been measured *under*. That is the thesis of the project, applied to ourselves.

7.1 An attack that succeeded by surrendering the money

An early engine reported 100.0% success — achieved by shrinking transactions to roughly 12% of their value. An attacker who gives up 88% of the take to avoid detection has not defeated the system; they have been priced out of it. Adding an economic floor moved retained attacker value from 11.4% to 74.5%, and changed the attack's shape: the flawed version reached for the amount lever in 57 of 60 successes, the corrected one spreads across amount, timing and merchant choice.

7.2 A missing measurement rendered as a confident zero

The dashboard plotted detector quality per round and substituted `0` where a round had not been trained. It drew detection quality collapsing to the axis, directly beneath a caption saying detection quality holds — with the correct value displayed two inches away. The cause was one defaulting expression, `??  0`, which converts *"I have no measurement"* into *"I measured zero"*.

A missing measurement presented as a confident zero is not a smaller error than a fabricated result. It is a fabricated result, produced automatically. The fix was not a better default but the removal of the default: the value is nullable, absent rounds are drawn absent, and the table names them "not run".

7.3 A threshold fitted on the test split

The loop chose its operating threshold by maximising F1 on the test split — contradicting our own documented policy, and fitting the operating point on the rows the attack was then scored against. It lifted the bar to roughly 0.94 against a budget-derived cut near 0.23, and raised it further every round, so each round of "defence" quietly handed the attacker an easier target than the last.

The honest part is the outcome: fixing it made evasion genuinely harder, and attack success stayed at 100.0% anyway. The bug was not propping up the result. Had we found it after publishing, the number would have survived — but we would have been quoting it for the wrong reason and could not have told the difference.

7.4 We refuted our own excuse

The draft explained the flat attack success rate as an artefact of small adversarial dosage. Figure 3 is the test of that explanation, and it fails: 5000× the dosage changes nothing and costs 22.3% of PR-AUC. We deleted the excuse rather than keeping it as a hedge.

7.5 A fifth, caught while writing this document

The artifacts moved underneath this writeup mid-draft. An earlier run had mean features touched rising 3.81 → 4.64; the current one has 4.12 → 4.03. The prose, the figure and the prototype all said attacker effort was climbing, and one of the two axes had stopped agreeing.

It was caught mechanically rather than by reading: this document defines every quoted value as a macro, and a cross-check of defined-versus-referenced macros flagged one that had gone unused. The template also hard-coded a `+` before the delta, so a fall would have rendered as "+-2%" — a sign error announcing a substantive one. The deltas now carry their own sign, and the figure plots the axis that actually moved.

The claim that survives is narrower than the one we had written. Retraining makes evasions harder to *find* — median queries 275 → 391 — and does not make them structurally more expensive.

7.6 Two smaller retractions, same species

- We reported the static export as working from `file://`. It resolved every asset and rendered blank charts, because a cross-origin chunk fails a CORS check against an opaque origin. *Assets resolving* and *a page working* are different claims and we had checked the first.
- The live agent demo opened on an injection/scenario pair where nothing happened in either condition — only 7 of 144 pairs exploit gpt-oss-120b undefended. A judge would have concluded the demo was broken rather than that the model held. It now opens on a pair that actually exploits.

8 Reproducibility

The same logic exists in two languages three separate times, and each pair is held equal by a check that fails loudly rather than by good intentions.

Guarantee	Covers	Command
Artifact contract cannot drift	10 mirrored interfaces	`pytest tests/test_artifacts.py`
Live agent runs the corpus defences	144 documents, 360 spans	`npm run check:agent`
Browser detector is the trained model	68 rows, agreement to 10^{-6}	`npm run check:trees`
Fixture data announces itself	14 enveloped artifacts	`pytest`

Every artifact carries a placeholder flag, a git SHA and a timestamp. While any figure on the prototype is seeded fixture data, a banner names the exact files — so no number on the site can quietly be invented. Every figure in this document is generated by `scripts/build_figures.py` from those same artifacts.

9 Limitations

- Attack success does not fall. We report the cost imposed instead, and we tested and rejected the two obvious explanations for why it does not fall.
- The adversarial-detection result generalises to unseen instances of a shape the model has seen. It is not evidence against a fresh adaptive search.
- The agentic result is significant on one vendor and on the pooled corpus, and not on the other vendor. All three are reported.
- The corpus is a public simulation of card transactions, not production data from a live rail.